P01-10 · Execution Script — T1 to T20, Day by Day, Prompt by Prompt

This is the page you actually work from. Ten working days, twenty tasks, one prompt each, in strict order. Every prompt names the role hat, points at the page holding the frozen code, states the stop point, and demands pasted evidence before the next one starts. **Do not run two prompts in one session.**

1. The ten-day schedule

Day	Tasks	Source page	Gate that must be green before the next day
1	T1	P01-01	`cargo build --workspace` • `./scripts/check-banned.sh` • `cargo xtask contracts`
2	T2, T3	P01-02	`cargo test -p aura-core`; every code has a `user_message` with no jargon
3	T4, T5	P01-03	Schema applies; `integrity_check` clean; `STRICT` on every table
4	T6	P01-03	Second instance refused with `AURA-DB-3005`; migration backup verified
5	T7, T8	P01-04	Scan order deterministic; hash throughput measured on real files

Day	Tasks	Source page	Gate that must be green before the next day
6	T9, T10	P01-04	Second import of the same folder imports 0 files; digest identical
7	T11, T12	P01-05	All 8 kill points converge to the same catalog digest
8	T13, T14, T15	P01-06	App launches on 3 platforms; IPC type parity; cancel under 500 ms
9	T16, T17	P01-07	Hostile corpus passes; coverage at or above 85 %; zero flakes in 10 runs
10	T18, T19, T20	P01-08, P01-09	CI green; all 28 evidence rows filled; exit report signed

Two weeks is planned, ten working days is the schedule. The slack is deliberate: days 5–7 are the ones that historically overrun, because that is where real filesystems and real crashes meet the design.

2. Session opener — paste this once at the start of every session

> You are building AURA, an offline-first wedding photo AI for professional photographers, in a Tauri 2 + React 18 + TypeScript shell over a Rust 2021 core. We are in PHASE-01 of 30: Project Foundation, Catalog and Wedding Project Ingest.

> Standing rules for every task in this phase, without exception:

> 1. **No silent failure.** Every failure path returns an `AuraError` with a registered code. No `unwrap`, no `expect` outside tests, no ignored `Result`.

> 2. **Every run resumable.** Assume the process is killed with no warning at any instruction. Work in transactions; never leave a half state.

> 3. **Same input, same bytes.** No `HashMap` iteration in anything ordered, no unseeded randomness, no `SystemTime::now()` outside the `Clock` trait.

> 4. **Budgets are assertions.** A budget lives in `perf/budgets.toml` and fails a test when exceeded.

> 5. **Quality is a gate.** If the acceptance criteria are not met, the task is not done, regardless of how much code exists.

> 6. **RAW files are never modified.** Open read-only. Every write goes to the catalog or a sidecar.

> 7. **No network calls in phase 01 at all.** Not for telemetry, not for updates, not for fonts.

> 8. **No client path, name or image in any log, error message or telemetry.**

> When a page says a contract is FROZEN, copy it **verbatim**, including comments. Do not improve it, rename fields, reorder variants or change types. Those comments explain why an obvious-looking simplification is wrong, and the file's BLAKE3 digest is checked in CI.

> When you finish a task: stop, run the exact commands listed, and paste the real output. Do not start the next task. Do not summarise output as "all tests pass" — paste it.

> If a specification is ambiguous or wrong, say so and ask before inventing. A wrong guess frozen into a contract costs the next 29 phases.

3. The twenty prompts

Day 1 — T1 Repo scaffold, toolchain and the banned-pattern gate

> Act as `DEVOPS` with `CTO` reviewing. Task T1, from P01-01.

> Create the repository skeleton: workspace `Cargo.toml` with all 9 members and the pinned `[workspace.dependencies]` block, `rust-toolchain.toml` at 1.83.0 with the three targets, `rustfmt.toml`, `clippy.toml` with the `disallowed-methods` and `disallowed-types` entries, `deny.toml`, `.gitignore`, `.githooks/pre-commit`, the crate-root lint block in every crate, `scripts/check-banned.sh`, and `xtask/src/main.rs` with the BLAKE3

> contract-digest checker writing `contracts.lock`. Add the `no_upward_deps` test. Scaffold `ui/` with Vite, React 18 and TypeScript strict.

> Constraints: every dependency version is pinned in the workspace root and inherited with `workspace = true`; `panic = "abort"` in release; `#![forbid(unsafe_code)]` in every crate except where an FFI crate later needs it; the banned-pattern script must be a hard failure, not a warning, and must support the `// ALLOW-BANNED:` escape hatch with a required reason.

> Stop after T1. Paste: `cargo build --workspace --all-targets`, `cargo clippy --workspace --all-targets -- -D warnings`, `./scripts/check-banned.sh`, `cargo xtask contracts`, `git config core.hooksPath .githooks` output, and the generated `contracts.lock`. Then add an `unwrap()` to one file, paste the failing gate output, and revert it.

Day 2 — T2 Error taxonomy and the code registry

> Act as `TLC`. Task T2, from P01-02.

> Copy `crates/aura-core/src/contract/error.rs` verbatim: `Severity`, `Recovery`, `ErrorCode` with `domain()` and `runbook_url()`, `AuraError` with `with_context`, `with_cause`, `is_retryable`, `stops_run`, and `AuraResult<T>`. Create `errors.toml` with all 19 phase-01 codes. Write the two registry tests, including the one asserting no `user_message` contains a technical token. Write `errors/io.rs`, `errors/db.rs` and `errors/job.rs` mappers from OS and SQLite errors to codes, and `redact.rs`.

> Constraints: `AuraError` is constructible only through the registry, never with a free-form string; `detail` and `context` are for logs and never shown to a user; redaction happens at the moment of capture, not at the moment of writing, so an unredacted value never exists in memory long enough to be logged by accident.

> Stop after T2. Paste `cargo test -p aura-core` in full, and the rendered table of all 19 codes with their severity, recovery and user message so I can read every sentence a photographer might see.

Day 2 — T3 Typed ids, clock, paths and consent

> Act as `TLC` with `SEC` reviewing. Task T3, from P01-02.

> Copy `contract/ids.rs` (the `typed_id!` macro, `ProjectId`, `PhotoId`, `FileId`, `RunId`, `ImportId`, UUID v7, `ContentHash`) and `contract/consent.rs` verbatim. Implement `clock.rs` with `Clock`, `SystemClock` and `FixedClock` including `advance_ms`. Implement `paths.rs` with `AppPaths::resolve`, the 15 `CLOUD_MARKERS`, `assert_not_cloud_synced`, and the Windows `\\?\` prefix above 240 characters.

> Constraints: ids carry their prefix in the string form so a `PhotoId` can never be passed where a `FileId` belongs, and mixing them must be a compile error — prove it with a `compile_fail` doc test; `ProjectConsent` has exactly one constructor and it is `denied()`; nothing in the crate may call `SystemTime::now()` outside `SystemClock`, and the banned-pattern gate must enforce that.

> Stop after T3. Paste `cargo test -p aura-core`, the `compile_fail` test output, and `AppPaths::resolve` output on this machine.

Day 3 — T4 Schema v1

> Act as `SRC`. Task T4, from P01-03.

> Create `migrations/0001_init.sql` verbatim: every table, every `CHECK`, every index, the `v_project_counts` view. Write `crates/aura-catalog/tests/schema.rs` asserting `STRICT` on every table, that every enum column has a `CHECK`, that every foreign key has an index, and that the schema matches the committed golden file.

> Constraints: every table is `STRICT`; every timestamp is RFC-3339 UTC text; no boolean columns — checkboxes are `'__YES__'` / `'__NO__'` text; `photo_file` carries the `UNIQUE (project_id, content_hash, rel_path)` and `UNIQUE (root_id, rel_path)` constraints exactly as written, because idempotent import depends on both.

> Stop after T4. Paste the test output, `PRAGMA integrity_check`, and the full `.schema` dump of a freshly created catalog.

Day 3 — T5 Pragmas, single writer and read pool

> Act as `SRC` with `PERF` reviewing. Task T5, from P01-03.

> Implement `db.rs::apply_pragmas` with the exact values from the page, `writer.rs` as the single-writer actor over a `crossbeam_channel::bounded(256)` on a thread named

> `aura-catalog-writer`, with `with` and `transact` using `TransactionBehavior::Immediate`, and the four-connection read pool with `query_only = ON`.

> Constraints: `synchronous = FULL`, not NORMAL — the comment explaining why stays in the code; exactly one connection in the process may write, enforced by the type system, not by convention; a panic inside a writer closure must not poison the actor for subsequent callers.

> Stop after T5. Paste `cargo test -p aura-catalog`, the pragma readback from a live connection proving every value took effect, and the concurrency test showing 8 threads writing through the actor with zero `SQLITE_BUSY`.

Day 4 — T6 Migrations, lock and verified backup

> Act as `SRC`. Task T6, from P01-03.

> Implement `guard.rs::CatalogLock::acquire` using an `fs4` advisory exclusive lock on `<catalog>.lock` returning `AURA-DB-3005`, `migrate.rs` with `APP_SCHEMA_VERSION = 1`, embedded migrations via `include_str!`, refusal on a newer schema, and a verified backup before any migration. Implement `backup.rs::create_verified` and `prune(dir, 5)`. Wire the six-step `open()` refusal chain in exactly the specified order.

> Constraints: no PID files — an advisory lock is released by the OS when a process dies, a PID file lies after a crash; a migration never runs without a backup that has passed `integrity_check` and a row-count comparison; the schema-too-new case refuses to open rather than attempting a downgrade.

> Stop after T6. Paste `cargo test -p aura-catalog` in full; the output of launching a second process against the same catalog; the `backups/` directory listing after a migration; and the six-step refusal chain exercised once per step, with each code visible.

Day 5 — T7 Scan

> Act as `SRC`. Task T7, from P01-04.

> Implement `scan.rs`: the extension sets, `classify()`, the 13 `SKIP_DIRS`, `ScannedFile`, `ScanOutcome`, and `scan_root` with `follow_links(false)`, `sort_by_file_name()`,

> `max_depth(32)`, the `._` and `.DS_Store` skips, zero-byte and long-path rejection, and the settle-window skip. Implement `util.rs` with `compare_key` using NFC, `case_fold_key`, `mtime_ms` and `rel_path_forward_slash`.

> Constraints: `sort_by_file_name()` is load-bearing for determinism, not cosmetic — keep the `// DETERMINISM:` comment; a symlink is never followed; a file modified inside the settle window is skipped this run rather than read half-written; scanning must terminate on a symlink loop.

> Stop after T7. Paste `cargo test -p aura-ingest scan`, the scan of the hostile corpus with every skip and rejection reason listed, and proof that two scans of the same tree produce identical ordering.

Day 5 — T8 Hashing

> Act as `SRC` with `PERF`. Task T8, from P01-04.

> Implement `hash.rs`: full-content BLAKE3 with a 1 MiB buffer, size-mismatch detection returning `AURA-IO-1007`, and `hash_batch` with an explicit rayon pool sized from the storage-type table.

> Constraints: hashing streams and never reads a whole file into memory — prove it with the 200 MB hostile fixture and an RSS assertion; a file whose size changes between `stat` and end-of-read is a retryable error, not a recorded hash; thread count comes from the storage table, because 8 threads on a spinning disk is slower than 1.

> Stop after T8. Paste `cargo test -p aura-ingest hash`, the measured MB/s on this machine with the thread count used, and the peak RSS while hashing the 200 MB fixture.

Day 6 — T9 Pairing and EXIF

> Act as `SRC`. Task T9, from P01-04.

> Implement `pair.rs` with `PhotoGroup` and `group()` using BTreeMap buckets keyed on (directory, case-folded stem), the double-extension strip for `IMG.CR2.xmp`, and the RAW-primary bonus. Implement `exif.rs` with `ExifFacts` and `extract()` capped at 2 MiB, 256-character string truncation, sorted deduplicated `raw_tags`,

> integer microsecond exposure, `DateTimeOriginal` then `DateTimeDigitized` precedence, and GPS range validation.

> Constraints: a parse failure is a Warning that still imports the file — never a lost photo; `capture_time_source` is recorded on every photo because PHASE-26 multi-camera matching depends on knowing whether a timestamp came from EXIF or from the filesystem; an orphan sidecar is retained, never discarded.

> Stop after T9. Paste `cargo test -p aura-ingest pair exif`, the grouping of the hostile corpus, and the extracted facts for all seven adversarial EXIF fixtures showing that each degraded rather than failed.

Day 6 — T10 Idempotent import

> Act as `SRC` with `TLC` reviewing. Task T10, from P01-04.

> Implement `import.rs`: the seven-step algorithm, the `fast_key` skip pass, 200-file batches each in one `IMMEDIATE` transaction, counters updated inside the same transaction, `mark_absent_unseen`, and the quarantine path with per-code UI messages. Use the `INSERT ... ON CONFLICT (root_id, rel_path) DO UPDATE ... RETURNING file_id` statement exactly as written.

> Constraints: importing the same folder twice imports zero files and produces a byte-identical catalog digest; a copy of the same folder at a different path adds file rows but not photo rows; counters can never disagree with the rows they count, which is why they are written in the same transaction; a quarantined file never blocks the run.

> Stop after T10. Paste `cargo test -p aura-ingest` in full; the import report for the Small corpus; the report for the immediate re-import showing 0 imported; the two catalog digests side by side; and the photo-versus-file counts for the duplicated folder.

Day 7 — T11 Job graph and leases

> Act as `TLC`. Task T11, from P01-05.

> Copy `contract/graph.rs` verbatim: `TaskId`, `TaskState` with `is_terminal()` and `can_transition_to()`, `CostHint`, `Lease`, the three constants, `WorkerId`, `Task`, `TaskSpec`.

> Implement `store.rs`: `enqueue` with whole-batch validation before any write, `claim_next` as a single `RETURNING` statement with the specified ordering and budget filter, `heartbeat` returning false when reclaimed, and `complete` checking the transition then promoting or skipping dependents in the same transaction.

> Constraints: `can_transition_to` is the single source of truth for legality and every mutation consults it; claiming is one statement, because a select-then-update lets two workers claim the same task; a whole `enqueue` batch is validated before any row is written, so a cyclic dependency cannot leave half a graph behind.

> Stop after T11. Paste `cargo test -p aura-jobs graph`, the state-transition matrix printed from `can_transition_to`, and a 16-thread claim test proving no task is claimed twice.

Day 7 — T12 Reclaim, worker and crash resume

> Act as `TLC` with `QAL` reviewing. Task T12, from P01-05.

> Implement `reclaim.rs` with `reclaim_expired` and `recover_on_startup` including the consistency audit and `AURA-JOB-7005`, `ResumeSummary`, and `worker.rs` with the `StageHandler` trait, 250 ms cancel checks, the heartbeat thread, and the lost-lease result discard. Wire the phase-01 stage graph `scan → hash → index` with the three priorities. Write the full chaos test with all 8 kill points.

> Constraints: `recover_on_startup` ignores lease expiry entirely, because a lease from a dead process is meaningless regardless of its clock; a worker that discovers it lost its lease discards its result rather than writing it; every one of the 8 kill points must converge to the identical catalog digest, and the test asserts the digest, not merely the count.

> Stop after T12. Paste `cargo test -p aura-jobs` in full; the chaos test output listing all 8 kill points with their resume times and digests; and the `ResumeSummary` from one real kill-and-relaunch cycle.

Day 8 — T13 Tauri shell and IPC v1

> Act as `SFE` with `MBE`. Task T13, from P01-06.

> Copy `crates/aura-app/src/ipc/contract.rs` and `ui/src/ipc/types.ts` verbatim. Write the `cargo xtask ipc-types` generator that reproduces the TypeScript from the Rust and wire `--check` into the PR lane. Implement `commands.rs` for all 14 commands with the single `From<AuraError> for IpcError` conversion, the typed `call` wrapper, and `tauri.conf.json` with the exact CSP.

> Constraints: no command blocks the UI thread; no error reaches the UI as a bare string; `detail` and `context` never cross the boundary; the CSP contains no `https:` in `connect-src`, so a stray `fetch` fails at runtime.

> Stop after T13. Paste `cargo test -p aura-app`, the command-parity and types-parity test output, and the app launching on this platform with one successful command round trip visible in the console.

Day 8 — T14 First run and consent

> Act as `SFE` with `SEC` reviewing. Task T14, from P01-06.

> Implement the first-run flow: welcome, catalog location with all four refusals and their exact wording, first project creation, and the consent screen with every toggle defaulting off and no Skip button.

> Constraints: the four refusals use the codes and the exact sentences on P01-06 — do not reword them; a removable volume warns and can be accepted deliberately; a new project's consent is all-false after a database round trip, not merely in memory; no network call anywhere in this flow.

> Stop after T14. Paste the four refusal tests, the consent round-trip test, and screenshots or a recording of all four refusals firing.

Day 8 — T15 Honest progress and the grid

> Act as `SFE`. Task T15, from P01-06.

> Implement `progress.rs` with the monotonic guard, 10 Hz throttle and the three ETA gates; `useProgress` with the animation-frame coalescer; and `GridScreen` virtualised for 4,000 cells.

> Constraints: progress never decreases, even when a retry recounts; an ETA that fails any gate is `None` and the UI renders nothing rather than "calculating"; `current_item` is a bare file name, never a path, because progress events reach the log; a progress tick must not re-render 4,000 cells.

> Stop after T15. Paste the monotonicity, ETA-gate, throttle and file-name tests; the cancel-latency measurement; and a 10-second scroll trace summary for 4,000 cells with the worst frame time.

Day 9 — T16 Fixture generator

> Act as `QAL` with `DATA`. Task T16, from P01-07.

> Implement `cargo xtask fixtures` with the SplitMix64 generator, all four corpora, the synthetic wedding day with three camera bodies and one hour-skewed clock, and the complete 60-file hostile corpus with its `manifest.toml` declaring the expected outcome of every file.

> Constraints: fixtures are generated, never committed; the same seed produces byte-identical trees on all three platforms; the manifest declares expectations up front so tests assert against a declared truth rather than against observed behaviour; synthetic RAWs are deliberately not decodable, and that limitation is recorded.

> Stop after T16. Paste the generation output for all four corpora with timings and sizes; the tree digest from this platform; and the full `manifest.toml` for the hostile corpus.

Day 9 — T17 The full test suite

> Act as `QAL` with `QAIQ` reviewing. Task T17, from P01-07.

> Write every test in sections 4 through 8 of P01-07: 14 property tests, 11 chaos scenarios with real breakage, the golden manifest, the privacy and security tests, and the five hygiene meta-tests. Write out every elided body in full.

> Constraints: no `thread::sleep` outside chaos; every test asserts on a value, not on `is_ok()`; every `#[ignore]` carries a reason; every registered code is asserted

> somewhere; the volume-yank and disk-full tests use real mounts and a real full filesystem.

> Then break three things deliberately — the NFD comparison, the progress monotonicity guard, and path redaction — and paste the three failing outputs before reverting.

> Stop after T17. Paste `cargo test --workspace` in full; the chaos lane with all 11 scenarios and timings; the coverage report; and the three deliberate-failure outputs.

Day 10 — T18 CI and budgets

> Act as `DEVOPS` with `PERF`. Task T18, from P01-08.

> Create `perf/budgets.toml`, `crates/aura-perf/src/budget.rs` with the three-outcome `assert_budget` and real per-platform `peak_rss_mb`, all ten budget tests, `scripts/check.sh`, both workflow files, and `cargo xtask perf-compare`.

> Constraints: budgets live in exactly one file; a ceiling breach fails the build; every perf test asserts the work happened before it asserts the timing; CI stage 2 does not start until stage 1 passes; branch protection requires exactly one aggregate check.

> Then prove all six gates can fail: mis-format a file, add an `unwrap`, edit a frozen contract, hand-edit `types.ts`, comment out a test file, set a budget to an impossible value. Paste all six failures, then revert.

> Stop after T18. Paste a green PR-lane run with total wall time, the six deliberate failures, `peak_rss_mb` on this platform, and the first `perf/baseline.json`.

Day 10 — T19 Runbooks

> Act as `DOC` with `SEC` reviewing. Task T19, from P01-09.

> Write all 19 error-code runbooks following the `AURA-IO-1003` template exactly, plus the four `OPS-*` runbooks. Implement `cargo xtask runbooks --check`. Implement `aura-cli diagnostics` producing the seven-file redacted bundle.

> Constraints: every runbook opens with what the photographer sees, in their language; no Rust type, crate name or stack trace appears above the "For support and engineers" heading; the diagnostics bundle has no unredacted mode; front matter must agree with `errors.toml`.

> Stop after T19. Paste the `runbooks --check` output listing all 19; the failure output for a deleted runbook and for a runbook with an unregistered code; and the complete file listing plus contents of a real diagnostics bundle so I can confirm nothing client-identifying is inside.

Day 10 — T20 Evidence pack and exit report

> Act as `QAL` with `PM`, and get sign-off from every role named on P01-09. Task T20, from P01-09.

> Create `PHASE-01-EVIDENCE.md` with all 28 rows, `scripts/collect-evidence.sh`, `cargo xtask evidence --check`, the 8 ADRs, and `docs/phase-exit/PHASE-01.md` filled in from the template. Run the collection script and fill every row you can from this machine.

> Constraints: every artefact records host, commit and UTC timestamp; evidence from a different commit is rejected; a row you cannot fill is marked `PENDING` with the exact command and the named person who must run it — never invent a number; the manual 15-step walkthrough is done by a human on each platform and cannot be simulated.

> Stop after T20. Paste the `evidence --check` output; the filled `PHASE-01.md` with every measured budget in place; the list of `PENDING` rows with owners; and the eight ADRs. Then state plainly whether PHASE-01 passes its Definition of Done, and if not, exactly which rows block it.

4. Stop conditions — when to halt and ask rather than continue

Signal	Why it is a hard stop	What to do
A frozen contract "needs" a change	Seven contracts are inherited by 29 later phases; a silent edit	Stop, write the ADR, get it approved, then change the

Signal	Why it is a hard stop	What to do
	invalidates every downstream assumption	contract and the digest in one commit
A test is flaky and the cause is unclear	An intermittent failure in a resume path is usually a real race, not a flaky test	Stop and find it. Do not add a retry or a sleep
A budget is missed by more than 30 %	That is a design problem, not a tuning problem	Stop and review the approach before optimising
Two kill points produce different digests	The resume guarantee is broken, which is the core promise of the phase	Stop everything. Nothing else in phase 01 matters until this converges
A path appears in a log	A privacy leak reaching main is a breach, not a bug	Stop, fix redaction at the capture point, add the assertion, then continue
A spec on these pages is wrong	Silently working around it means the page and the code disagree from day one	Say so, propose the correction, update the page, then implement

5. Daily close-out, five minutes

- [ ] `./scripts/check.sh` green locally
- [ ] Every acceptance box for today's tasks ticked with pasted evidence, not from memory
- [ ] New error codes added to `errors.toml` **and** given a runbook the same day
- [ ] Any new frozen contract added to `contracts.lock` in the same commit
- [ ] Anything surprising written down as an ADR while the reasoning is still fresh
- [ ] Tomorrow's first prompt already open, with the session opener pasted above it

6. What "phase 01 is done" means, in one sentence

Point AURA at 4,000 RAW files on a card, kill the process at any moment during the import, relaunch, and reach a complete deduplicated catalog with zero duplicate rows, honest progress throughout, every failure carrying a code with a

runbook, no client path in any log, and every budget measured and inside its ceiling on the reference machine — with the evidence pasted into the exit report and signed off by all eight roles.